2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA) | 979-8-3503-4235-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/SEAA60479.2023.00016

Autonomously Adaptive Machine Learning Systems: Experimentation-Driven Open-Source Pipeline

1st Yumo Luo
University of Helsinki, Finland
yumo.luo@helsinki.fi

2nd Mikko Raatikainen
University of Helsinki, Finland
mikko.raatikainen@helsinki.fi

3rd Jukka K. Nurminen
University of Helsinki, Finland
jukka.k.nurminen@helsinki.fi

***Abstract*—Machine Learning Operations (MLOps), derived from DevOps, aims to unify the development, deployment, and maintenance of machine learning (ML) models. Continuous training (CT) automatically retrains ML models, and continuous deployment (CD) automatically deploys the retrained models to production. CT and CD are essential for maintaining ML model performance in dynamic production environments and, therefore, need to be considered when practicing MLOps. We present our CTCD-e MLOps pipeline, implemented mostly using existing open-source software, being able to autonomously adapt an ML system to changing production environments by enabling flexible model CT and CD. The pipeline can automatically trigger a model retraining round when the model performance degrades. Then it automatically conducts an A/B test for the retrained model and its predecessor in production to start serving the better one. The pipeline was evaluated by two experiments. In the pipeline, users can flexibly configure the model retraining, as well as the redeployment and production A/B test of the retrained models based on various requirements.**

***Index Terms*—MLOps, continuous training, continuous deployment, A/B testing, open-source, experimentation, pipeline**

I. Introduction

Machine Learning Operations (MLOps) [1], derived from DevOps, is a collection of practices and techniques to unify the development, deployment, and operation of *machine learning (ML)* systems, aiming to streamline the process of shifting ML solutions to production. MLOps covers multiple phases of an ML project, such as data engineering, model training and evaluation, model validation, model deployment and serving, and model monitoring. An *MLOps pipeline* brings tool automation into MLOps [2] by multiple pipeline components responsible for different MLOps phases. As the production environment inevitably changes or cannot be covered comprehensively, it significantly affects ML models' performance as they are sensitive to dynamic data – models which have passed initial validation tests may not perform as expected in the long run. For example, a recommender system may fail to provide accurate recommendations due to the new items it has not been trained on before or because the customers' preferences have changed. Therefore, an ML model needs retraining to maintain its performance.

Especially when changes are frequent or uncertainty is high, *continuous training (CT)* and *continuous deployment (CD)* are needed for ML models. CT enables automatic model retraining, and CD automatically deploys retrained models to production. They enable ML systems to respond to changes in production by keeping models fresh. Retraining can be triggered periodically or by model monitoring results or repository updates [3]. A commonly used strategy during CD in today's software engineering is *A/B testing*, meaning experimenting with a redeployed model on a small percentage of user traffic. A/B testing can validate the performance of a retrained model in production and mitigate the risk of deploying a poorly performing model, further elevating the effectiveness of model CT and CD.

Proprietary and open-source solutions are available for developing MLOps pipelines with CT and CD support for models. However, proprietary solutions suffer from a lack of transparency and potential vendor lock-in, while existing open-source solutions may not offer flexible model retraining capabilities as elaborated on in SectionII. Due to these limitations, we identified an opportunity for a solution, implementable by open-source software, which provides ML models with flexible CT and CD support.

This paper presents an open-source and portable MLOps pipeline called *CTCD-e* (continuous-training-and-continuous-deployment-enabling), a practical solution to autonomously adapt ML systems to changing data by providing flexible CT and CD support for models. We designed a conceptual model of our MLOps pipeline and implemented it utilizing open-source tools as far as reasonable [1]. The pipeline can automatically start to retrain a model when its performance degrades and automatically A/B test the retrained model against its predecessor in production. The pipeline was evaluated by two experiments, which also highlight future research directions in CT/CD-aware MLOps pipelines.

This paper is structured as follows. The previous work is in Section II. We present the conceptual model of the CTCD-e MLOps pipeline in Section III and its implementation walkthrough in Section IV. We describe the evaluation in Section V. We discuss our MLOps pipeline and further research directions in Section VI. Finally, we summarize the conclusions in Section VII.

We acknowledge the IML4E project of ITEA funded by Business Finland.

[1] Available at https://version.helsinki.fi/luoyumo/ctcd-e-mlops-pipeline

2376-9521/23/$31.00 ©2023 IEEE
DOI 10.1109/SEAA60479.2023.00016

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

II. Previous work

CT and CD have been considered essential operations for improving the automation level and, thereby, the maturity of MLOps. Google [4] presents a framework for accessing MLOps maturity. Level-0 MLOps has no automation, and model development and deployment are manual. In level-1 MLOps, a pipeline is established to bring automation, with model CT and CD being the important objectives of the pipeline. Level-2 MLOps, taking a further step from level-1, allows updating the pipeline components frequently. Two MLOps frameworks have been derived from literature reviews [5], [6], both of which recognize model CT and CD as necessary activities for adapting models to dynamic production environments. A literature review [3] outlines the retraining triggers used for model retraining and discusses the challenges of continuously deploying models to production. A reference architecture [7] has been proposed to retrain models using the new incoming data, and an MLOps framework called "CD4ML" (Continuous Delivery for Machine Learning) [8] has been introduced to trigger model retraining rounds based on the information collected from model monitoring. These two are high-level guidelines for implementing an MLOps pipeline with model CT and CD functionalities but lack concrete implementations. In an example implementation [2] for the CD4ML framework, the flow is single-directional, from model training to model monitoring, without retraining triggers from the model monitoring.

Proprietary pipelines exist with CT and CD support, especially from cloud vendors. For example, AWS SageMaker [9] enables users to build MLOps pipelines where data drift can trigger model retraining. Another example proprietary MLOps pipeline has been implemented to support deploying models to edge devices [10], where performance degradation triggers model retraining. The implementation relies on Azure services. However, proprietary pipelines have limitations, such as a lack of transparency and potential vendor lock-in.

Much effort has been dedicated to developing MLOps pipelines using and integrating the existing open-source tools designed for one or more specific activities or tasks of MLOps [11], [12]. However, the focus is on model training and deployment, ignoring model monitoring and lacking CT for models. In other words, these MLOps pipelines can automatically deploy new models, but model training rounds need to be manually triggered, e.g., by pull requests to a repository. Kubeflow Pipelines, TensorFlow Extended, Kedro, Metaflow, and ZenML are such open-source frameworks that can orchestrate MLOps pipelines. However, Kubeflow Pipelines, Kedro, and Metaflow focus on model training and deployment, resulting in MLOps pipelines that flow in a single direction from data to model training and finally to model deployment. Although TensorFlow Extended and ZenML integrate model monitoring but lack flexibility, only supporting periodic model retraining and retraining based on the arrival of new data. The periodic retraining may lack responsiveness to rapidly changing data, and using the arrival of new data as a retraining trigger may lead to unnecessary training rounds. Therefore, none of these open-source frameworks provides an end-to-end solution that supports flexible CT and CD for models.

[2]https://github.com/ThoughtWorksInc/cd4ml-workshop

III. Key Functionalities and Conceptual model

The key functionalities of the CTCD-e MLOps pipeline are presented in Table I. The software selection (the third column) to implement the functionalities is detailed in Section IV. The functionalities are based on the previous work outlined above, emphasizing CT and CD that autonomously adapt in an experimentation-driven manner. We assume data quality is covered as a part of a DataOps [13] pipeline.

Model training, deployment, and serving are elementary activities of an MLOps pipeline and prerequisites for model CT and CD, hence Func1 and Func4. These functionalities cover initial activities for shipping the first model to production but also activities needed for adapting to changing data, such as retraining and redeployment. Regarding Func1, training a model usually requires sequential steps, such as data cleaning, feature engineering, and the actual training to produce model artifacts partly covered by DataOps. Similarly, model deployment may consist of multiple steps, such as model validation. This multi-step nature necessitates task orchestration, as is depicted in Func1. The details of these steps are use-case-specific, so we do not specify them in detail within our scope.

Tracing model training is essential in MLOps to elevate it to systematic and traceable engineering practice. ML model performance can be affected by many factors, such as training data and tuning parameters. For this reason, Func2 is required to improve traceability and reproducibility [5]. To provide better model governance, a central artifact registry is needed to store generated model artifacts (Func3). Func2 and Func3 also enable version tracking unless covered otherwise, e.g., by DataOps.

Following the CD4ML framework, we monitor model performance as the retraining trigger (Func5). Model retraining must be automatically retriggered when model performance drops (Func6).

A retrained and redeployed model (cf. Func1 and Func4) must be assessed against its predecessor to determine whether the new model outperforms its predecessor in production (Func7). A/B testing can reduce the risk of selecting a worse model. Finally, as CT and CD consist of multiple phases, from model training to model comparison in production, a dashboard must provide pipeline users with an overview of the events related to model training and model performance comparison (Func8).

The resulting conceptual model of our CTCD-e MLOps pipeline, which maps the needed functionalities to activities and storages and their interaction implementable by open-source tools, is summarized in Figure 1. It consists of six connected composite activities and two storages to which the functionalities are mapped. The metadata store records model training metadata, and the model registry stores trained models. Models are served and monitored in production. The

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I
KEY FUNCTIONALITIES AND SOFTWARE SELECTION TO IMPLEMENT THE FUNCTIONALITIES.

Func	Description	Selected software
Func1	Training and deploying models and orchestrating tasks related to model training and deployment.	Kubeflow Pipelines
Func2	Tracing model training metadata.	MLflow
Func3	Storing model artifacts.	MLflow
Func4	Serving models so that their predictions are available via APIs.	KServe
Func5	Monitoring models for the *functional performance*, such as prediction accuracy, as well as the input data, such as data drift, which impacts the functional performance of a model, and the *service-level performance*, such as response time and CPU and memory usage.	Evidently, Prometheus, Grafana, self-implemented component (Monitor)
Func6	Triggering model training automatically when model performance decreases.	Self-implemented component (Retraining-Triggering Webhook)
Func7	A/B testing models in production so that the pipeline serves multiple models, such as an old and a retrained model, splits production traffic to the models, compares models' performance, and, after the comparison, deploys the better model to receive all traffic.	Iter8, self-implemented component (Model Comparison Runner)
Func8	Tracing events related to model training and performance A/B testing.	Self-implemented component (Info-Collector)

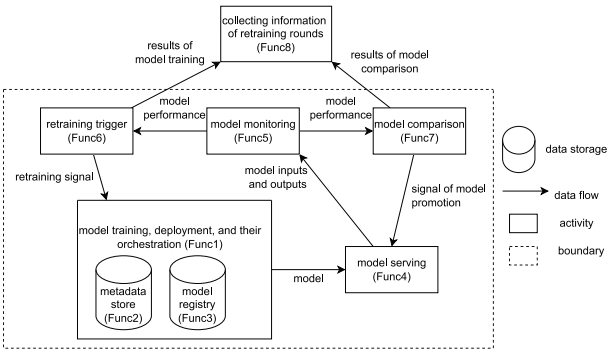

Fig. 1. Conceptual model of the CTCD-e MLOps pipeline. The functionalities are mapped to the activities and storages.

monitored metrics of models are used for determining model retraining and comparisons in A/B testing. After the comparison, the winning model, whether the retrained or the old one, is promoted, i.e., fully deployed to receive all production traffic, and the losing model is removed. The results of model training rounds and model performance comparisons are collected to provide users with an overview of what is happening in the pipeline.

IV. IMPLEMENTATION

In this section, we first outline our choices of the open-source tools used to implement the CTCD-e MLOps pipeline. We then present an architecture walkthrough of the pipeline implementation. The retraining trigger (Func6) and A/B testing (Func7) as key novel components of the pipeline are elaborated on after the architecture walkthrough.

A. Decisions on Software Selection

To mitigate the potential vendor lock-in associated with proprietary solutions, our MLOps pipeline needs to be portable, i.e., installable on various cloud platforms and on-premises environments. Therefore, we chose Kubernetes as the underlying platform for hosting our MLOps pipeline.

The pipeline's tool selection for accomplishing the required functions is summarized in Table I. To avoid reinventing the wheel, we used third-party open-source software whenever possible. For pipeline components where no suitable open-source software was directly available, we implemented them ourselves. Among the third-party software, *Kubeflow Pipelines*[3] offers an SDK that allows developers to create individual ML tasks, such as data preprocessing and model fitting, known as KFP components. These components can then be organized into an ML workflow, referred to as a KFP pipeline. *MLflow*[4] can be deployed as a service and exposes endpoints for saving model training metadata and model artifacts using a relational database and standard object storage, for which we selected *PostgreSQL*[5] and *MinIO*[6], respectively. *KServe*[7] is a model serving platform built atop Kubernetes. *Evidently*[8] is a Python library for calculating incoming data quality and model performance metrics. *Prometheus*[9] and *Grafana*[10] form a commonly used monitoring stack in Kubernetes, where the former stores the monitored metrics, and the latter visualizes the metrics. *Iter8* [14] is a Kubernetes release optimizer for comparing the performance of different versions of services in a controlled manner.

B. Architecture

The architecture of the CTCD-e MLOps pipeline and the interaction between the pipeline components are illustrated in Figure 2. Below, we describe how the functionalities in Figure 1 are implemented by presenting an architecture walkthrough starting from the scenario where a model is already running in production, i.e., deployed to KServe. The numbers enclosed in circles in the following paragraphs correspond to the numbers inside parentheses in Figure 2. In the figure,

[3]https://www.kubeflow.org/docs/components/pipelines/
[4]https://mlflow.org/docs/latest/index.html
[5]https://www.postgresql.org/
[6]https://github.com/minio/minio
[7]https://kserve.github.io/website/0.9/
[8]https://docs.evidentlyai.com/
[9]https://prometheus.io/docs/introduction/overview/
[10]https://grafana.com/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

the third-party open-source components are white, whereas self-created components are blue and assigned a simple name reflecting their task (Joiner, Monitor, etc., in the figure), which are used throughout the paper.

Model serving. KServe exposes each model as a separate inference service atop Kubernetes. When a client application (represented as "APP" in Figure 2) sends requests for predictions, the KServe inference service receives the requests and responses with predictions ①.

Model monitoring and retraining trigger. KServe is built atop Knative[11], which provides an event-driven architecture allowing the inference service to publish the requests and their corresponding prediction responses as events to a Knative's broker, i.e., a middleware service of Knative for receiving and distributing events ②. Joiner subscribes to the request and response events and thus can receive the requests and predictions sent by the inference service ③. In many cases, labels need to be collected from various sources, such as downstream applications that consume the model's predictions. Joiner provides an endpoint to enable pipeline users to send labels to Joiner based on their use cases ④. Joiner then combines requests, predictions, and labels and sends them to Monitor ⑤. Joiner also saves the requests and the corresponding labels to the MinIO storage service so that they can be used as training data in model retraining ⑥. The training data of the running model is already in the MinIO storage service. Monitor loads the training data of the running model from the MinIO storage service and receives production data from Joiner. Monitor compares the training data to the production data as well as calculates metrics related to inputs, e.g., data drift degree and the functional performance, e.g., prediction accuracy, of the model, using the Evidently library ⑦.

Monitor exposes the calculated metrics through an endpoint from which Prometheus can scrape these metrics ⑧. The inference service also exposes the model's service-level performance metrics to Prometheus ⑨. Grafana queries these metrics from Prometheus to visualize them. Prometheus evaluates these metrics at regular intervals. If Prometheus detects exceptions, e.g., functional performance metrics drop under the pre-configured threshold, it sends an alert to Retraining-Triggering Webhook ⑩. Upon receiving the alert, Retraining-Triggering Webhook triggers a new run of model training in Kubeflow Pipelines ⑪. Pipeline users need to build their own KFP pipelines for model retraining using the SDK provided by Kubeflow Pipelines and upload them to the MinIO storage service beforehand.

Model (re)training, (re)deployment, and their orchestration, metadata store, and model registry. During the retraining run in Kubeflow Pipelines, the production data is downloaded from the MinIO storage service and can be used to retrain the model, either on its own or together with old training data ⑫. The retraining data is then converted to a format Monitor can handle and saved back in the MinIO storage service ⑬. The resulting metadata of model training and generated model artifacts are pushed to the MLflow service ⑭ and further to the PostgreSQL database ⑮ and MinIO storage service ⑯, respectively. When Kubeflow Pipelines sends a model deployment request to KServe ⑰, KServe creates an inference service by fetching the model artifact from the MinIO storage service and loading the model inside a model serving runtime ⑱. When a new model version is trained and deployed to KServe (green in the figure to illustrate the new model instance) following steps ⑫–⑱ as described above, Retraining-Triggering Webhook informs the completion of the retraining to Model Comparison Runner ⑲.

Model comparison. Upon receiving a notice of a completed model retraining round, Model Comparison Runner calls Iter8 to trigger an experiment to A/B test the performance of the retrained model against the old model. During the experiment, requests are split and forwarded to both the old and the retrained models, and Joiner also sends inputs, outputs, and labels to Monitor adhering to steps ①–⑤. The functional and service-level performance metrics are collected from both inference services and saved to Prometheus as depicted above (⑧–⑨). Iter8 queries these metrics from Prometheus to determine the better model ⑳. During the experiment, Iter8 can use its traffic-splitting algorithms [14] to adjust the traffic-split ratio to shift more traffic toward the better model as the experiment progresses. The traffic-splitting algorithm is based on Bayesian learning and multi-armed bandits. When the A/B test is completed, Model Comparison Runner collects the experiment results from Iter8 and calls KServe to fully deploy the winner model so that the winner model can start serving all requests – KServe also terminates the losing model ㉑.

Information collection. Info-Collector collects the results of model training runs in Kubeflow Pipelines and model performance comparison from Retraining-Triggering Webhook and Model Comparison Runner (㉒ and ㉓, respectively). Info-Collector then saves the results into the PostgreSQL database ㉔ and displays them in a UI.

After the model performance A/B testing, the winner model continues serving client requests ① and is continuously monitored ②–⑦. A retraining round will be triggered again basically infinitely when the winner model's performance degrades due to changing requests, as depicted above. The Retraining-Triggering Webhook will not launch a new retraining iteration before the earlier one is completed.

C. Retraining Trigger

Model retraining rounds are triggered by the alerts sent from Prometheus. Retraining-Triggering Webhook is a specific webhook service written in Flask for receiving Prometheus alerts and calling Kubeflow Pipelines to start model retraining. Prometheus is configured to evaluate model performance metrics scraped from Monitor and generate alerts of deteriorated metrics, which are sent to Retraining-Triggering Webhook via HTTP requests. Upon receiving an alert, Retraining-Triggering Webhook starts a model retraining round by triggering a running of the corresponding KFP pipeline in Kubeflow Pipelines.

[11] https://knative.dev/docs/

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

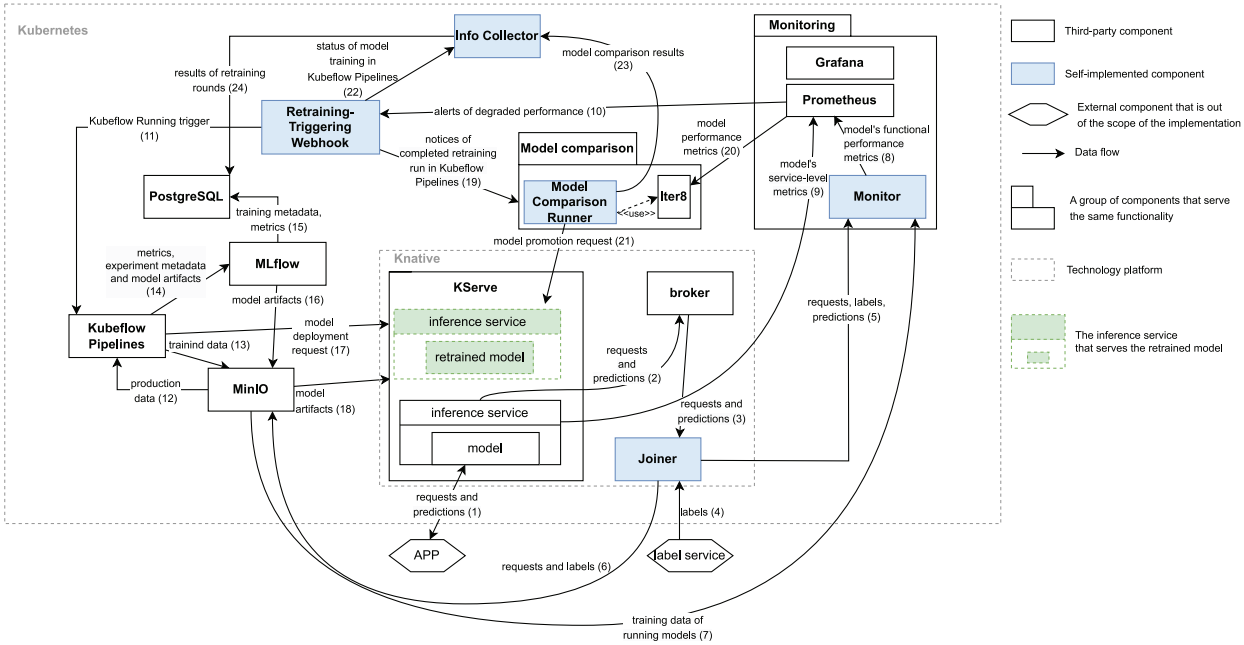

Fig. 2. Overview of the CTCD-e MLOps pipeline's architecture. Blue boxes represent self-created components, and white boxes are third-party open-source software. The green color denotes a served retrained model. Numbers inside parentheses refer to the given of the architecture walkthrough.

As previously mentioned, the KFP pipeline needs to be implemented and uploaded to the MinIO storage service beforehand. Retraining-Triggering Webhook configuration is specified in a YAML file, including the location of the retraining KFP pipeline of each model and the parameters (if any) required by the KFP pipeline. Retraining-Triggering Webhook downloads the KFP pipeline YAML files of all served models from the MinIO storage during the initialization. When it receives a Prometheus alert, it extracts the model name from the alert and uses the model name to find the corresponding KFP pipeline. Retraining-Triggering Webhook then spawns a new thread, where it uses the SDK of Kubeflow Pipelines to trigger a running of the corresponding KFP pipeline (i.e., a KFP run). After the KFP run is completed, Retraining-Triggering Webhook informs Model Comparison Runner to start A/B testing the retrained model against its predecessor.

D. Production A/B Testing

Model Comparison Runner is another dedicated Flask service of the CTCD-e MLOps pipeline responsible for automatically orchestrating A/B tests. For each A/B test, it collects the required information, generates a specification using the information, and employs the specification to invoke Iter8 to A/B test the performance of a retrained model against the old model. Three concepts used by Iter8 need to be explained to understand the functions of Model Comparison Runner.

- *Experiment*: Experiment is a custom resource definition provided by Iter8. It refers to a way of comparing the performance of two or more versions of a service. An experiment can consist of multiple iterations.
- *Reward metric*: The measured metrics from both services that determine the winning service.
- *Service-level objective (SLO)*: An SLO describes a performance level a service must achieve. A service loses A/B testing if it fails to meet an SLO, regardless of its reward metric.

To launch an experiment, Iter8 users need to create an experiment specification in a YAML file and use kubectl, the Kubernetes command-line tool, to apply the experiment specification to Iter8. An experiment specification declares the information needed for running an experiment, such as model versions involved in the experiment, SLOs, reward metrics, the number of experiment iterations, and the duration of each iteration. Model Comparison Runner integrates Iter8 into the CTCD-e MLOps pipeline by automatically creating experiment specifications and launching experiments on behalf of pipeline users. For each served model, an experiment configuration needs to be specified in the configuration YAML file of Model Comparison Runner in advance. An experiment configuration includes the above-mentioned information required for an experiment. When receiving a notification of a completed KFP run from Retraining-Triggering Webhook, Model Comparison Runner extracts necessary information from the notification, such as the model versions and the Kubernetes namespaces of the old and new models. Model Comparison Runner then uses the extracted information and the experiment configuration to create an experiment specification and apply it to Iter8.

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ALERT CONFIGURATIONS.

	Alert threshold	Alert pending time
Sine experiment	MAE >0.5	7 minutes
Wine experiment	MAE >0.3	30 seconds

Model Comparison Runner periodically checks if an experiment is completed. Once the experiment is completed, Model Comparison Runner collects the experiment results from Iter8 and uploads the results to the MinIO storage service so that pipeline users can check the results later. Model Comparison Runner reads the winner model from the results, requests KServe to remove the losing model, and modifies the routing rule of the winner model so that the winner model can receive all traffic.

V. EVALUATION

We evaluated the implemented CTCD-e MLOps pipeline's capability of retraining models and A/B testing the retrained models against the old models. In addition, we measured the memory consumption of the pipeline.

We used two cases to evaluate the CTCD-e MLOps pipeline's capability of model retraining and A/B testing: a simple, predictable, and mutable sine function ($sin(x) + n$) in which we changed n (*sine experiment*); and the simple but more realistic open red and white wine quality datasets[12] (*wine experiment*).

The pipeline components shown in Figure 2 were deployed to our local computers. Monitor calculated the mean absolute error (MAE) of the prediction and data drift, where data drift was calculated using statistical methods like the Kolmogorov–Smirnov test and Earth Mover's Distance. Prometheus scraped MAE values from Monitor every 25 seconds in both experiments. The threshold of triggering an alert and the pending time before Prometheus sent an alert to Retraining-Triggering Webhook in each experiment are shown in Table II.

The alert thresholds were determined based on pilot experiments with the models, where the prediction errors exceeded the thresholds upon changing the input data. Two different waiting durations were chosen to cover two scenarios: the short duration aims to promptly detect model performance degradation, while the long duration ensures sustained performance decline, thereby minimizing the risk of false positives due to temporary fluctuations. For A/B testing in both experiments, the 95th percentile response latency of the inference services was used as the SLO, requiring the latency not to exceed 5000 milliseconds. Regarding the reward metric, the smaller MAE, the better the model performance. Each A/B test consisted of three iterations, each lasting two minutes.

Before the experiments, we trained the models (a TensorFlow LSTM model in the sine experiment and a scikit-learn elastic-net model in the wine experiment) and deployed them to KServe. Requests were sent once a second in both experiments. To assess CT and CD functions in particular, we increased n in sine-function by one every 80 minutes. Respectively, white and red wine data were alternately sent to the KServe inference service, which naturally triggered retraining rounds because the model was trained with the wrong color wine data.

Both pipelines worked as expected, resulting in retrained models being served. No discontinuity of the pipeline was discovered during the experiment periods. The sine experiment's CTCD-e MLOps pipeline ran for 14 hours and triggered 12 retraining rounds, all of which yielded models with better prediction accuracy. Similarly, the wine experiment's CTCD-e MLOps pipeline ran for 24 hours, during which 26 retraining rounds were triggered. Among these retraining rounds, 11 resulted in a new model version replacing the old one. One possible reason not all retraining rounds yielded better models in the wine experiment could be that the color of wine data was changed again during some A/B testing rounds. As a result, the color of the wine data used for model retraining was different from the data the models received in these A/B testing rounds.

We also used the same setup of the wine experiment to monitor the memory consumption of the CTCD-e MLOps pipeline. Memory usage metrics of the pipeline components were collected by Prometheus until one retraining round and the corresponding A/B testing were completed. The metrics were then visualized in Grafana. We categorized the pipeline components to be measured into two groups: third-party components (i.e., Kubeflow Pipelines, KServe, Iter8, Mlflow, Prometheus, Grafana, the PostgreSQL database, and the MinIO storage service) and self-implemented components (i.e., Info-Collector, Model Comparison Runner, Joiner, Monitor, and Retraining-Triggering Webhook). As shown in Figure 3, Kubeflow Pipelines was found to be memory-intensive during the model (re)training phase, using up to 2GB memory (the red circle in the figure) to train an elastic-net regression model. Prometheus, together with Grafana, also required relatively much memory, around 2.5GB. As can be seen by comparing Figure 3 and Figure 4, the peak memory usage of the self-implemented components (the green circle in Figure 4) was small (less than 0.8GB) compared to Kubeflow Pipelines and the monitoring stack of Prometheus and Grafana, which may be mainly because these self-implemented components were deployed in single Kubernetes pods, whereas Kubeflow Pipelines and Prometheus had multiple pods. We also repeated the experiment by running two and four models in the pipeline. Figure 5 shows that the memory usage of Kubeflow Pipelines increased significantly with the number of models to be retrained at the same time, while the memory usage of Prometheus and Grafana was stable regardless of the increasing number of incoming requests due to the increasing number of models.

[12]https://archive.ics.uci.edu/ml/datasets/wine+quality

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

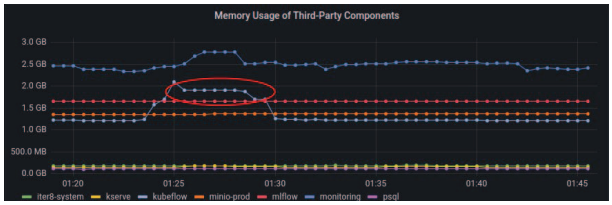

Fig. 3. Memory usage of the third-party components. Some third-party components, such as KServe and Kubeflow Pipelines, consist of multiple Kubernetes pods. To capture the memory usage of each component, we deployed them in separate namespaces, with the memory usage of a component being represented by the sum of memory usage across all the pods in its namespace. The labels represent the namespaces of the components. The "monitoring" label stands for Prometheus and Grafana.

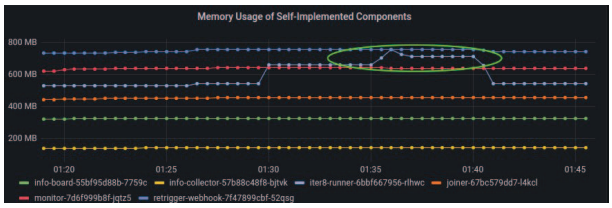

Fig. 4. Memory usage of the self-implemented components. Each component was deployed as a single pod except for Info-Collector, which has a backend pod and a frontend pod. The memory usage metric of each pod was collected. The labels represent the pod names, hence random strings in the labels due to the naming patterns of Kubernetes pods. The "info-board" label stands for the frontend of Info-Collector, "info-collector" for the backend of Info-Collector, and "iter8-runner" for Model Comparison Runner.

VI. Discussion

A. Discussion on the CTCD-e MLOps Pipeline

The presented end-to-end, iterative CTCD-e MLOps pipeline can autonomously adapt ML models to dynamic production environments by enabling model CT and CD. The pipeline is implemented entirely with open-source tools, most of which are existing third-party tools being more transparent than proprietary solutions. Due to Kubernetes, the MLOps pipeline can be deployed in on-premises and cloud environments, thereby mitigating vendor lock-in challenges when using proprietary solutions. Our CTCD-e MLOps pipeline is also generalizable to various ML frameworks: Kubeflow Pipelines is agnostic on ML frameworks, allowing pipeline users to choose any ML framework to implement their retraining. KServe is also capable of serving models trained using various ML frameworks.

Our pipeline also presents two notable novel features for facilitating ML model CT and CD and, thus, autonomously improving the adaptivity of an ML system. We developed and released open-source components to enable these features. First, Retraining-Triggering Webhook converts an alert generated by Prometheus into a call for Kubeflow Pipelines to start a model retraining round. This way, pipeline users can flexibly define the criteria for triggering model retraining using Prometheus' query language PromQL. Below is the PromQL query used in the wine experiment:

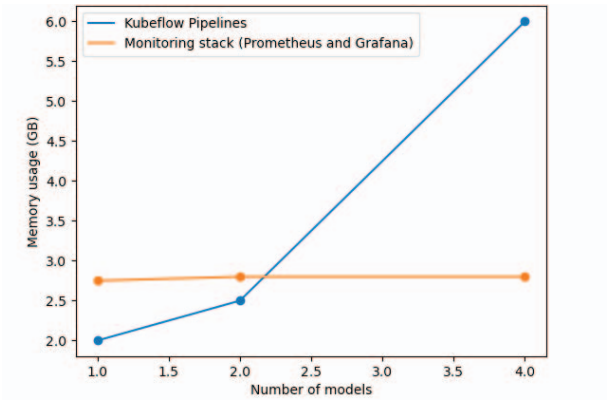

Fig. 5. Relationship between the number of models and the memory usage of Kubeflow Pipelines and the monitoring stack of Prometheus and Grafana.

$$(\text{evidently:regression_performance:quality}\{\text{dataset="current", metric="mean_abs_error", namespace="ns-baseline"}\} > 0.3) \text{ / on(dataset_name, namespace) (evidently:data_drift:dataset_drift } \{\} == 1.0)$$

Using PromQL, users can aggregate multiple model performance metrics and set specific thresholds for sending alerts. Additionally, users can specify the pending time for an alert, which may help avoid unnecessary model retraining if the performance degradation is only in a tolerably short period. Second, the Model Comparison Runner integrates Iter8 into the CD process, which enables pipeline users to A/B test the performance of a retrained model against its predecessor in production. The integration of Iter8 enables pipeline users to progressively roll out a better model and ensure that a retrained model can be fully deployed only when it outperforms its predecessor.

The pipeline can be extended to other deployment scenarios. Retraining-Triggering Webhook can be modified to provide an endpoint allowing users to manually start a new model training run in Kubeflow Pipelines. This way, users can manually deploy a new model version and A/B test it against the old version without needing to wait for the old one to deteriorate. Besides A/B testing, other common deployment strategies can be added. In a canary release strategy, the retrained model will be gradually rolled out as long as it satisfies the specified SLOs. Shadow deployment strategy involves directing all client traffic to the existing model while simultaneously copying it to the retrained model, which is particularly useful when the retrained model requires production-level validation before responding to client requests in production. These two strategies can be added to Model Comparison Runner.

The pipeline users need to configure and adapt the pipeline to their ML systems. Foremost, the users are required to define their own KFP pipelines for retraining their ML models. The selected Evidently library only supports tabular data. For other data types, Monitor can be re-designed to integrate user-defined monitoring methods. We implemented Joiner for passing model inputs, outputs, and ground truth to Monitor for further calculation of model performance metrics. Still, ground

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

truth must be collected from various resources. Collecting labels and sending them to Joiner are therefore left to pipeline users. An interesting option is to utilize an existing open-source feature store tool offering so advanced functionality that even replaces Joiner. Finally, users can configure the parameters related to model monitoring and performance A/B testing, such as the metrics to be monitored, the retraining trigger threshold, the SLOs and the number of iterations of each A/B test, and the duration of each iteration. In conclusion, our MLOps pipeline does not offer out-of-the-box automation for model CT or CD. However, compared to non-automated pipelines and systematic MLOps, the added effort remains moderate at most.

B. Challenges and Future Directions

Incorporating CT and CD based on model performance into our MLOps pipeline brought forward the following challenges that may inspire future research directions. Unlike the literature review [3] mentioned in Section II, which focuses on specific phases, the challenge we have encountered is related to the continuous approach as a whole.

Handling delayed or uncertain ground truth and manual labeling. Ground truth is required for monitoring model functional performance and preparing training data. A challenge is that ground truth may be delayed or uncertain, and labeling requires manual work in some use cases. Therefore, approaches for handling ground truth that is delayed or with different levels of uncertainty in model monitoring and model training need to be explored.

Diagnosing model performance deterioration and adaptive retraining. Besides detecting model performance issues, it is necessary to identify their underlying causes, which may affect how a model is retrained. For example, model performance can degrade because of outliers. In this case, the model can be retrained by including the outliers in a historical training dataset. Model performance can also degrade because the correlation between the inputs and the ground truth has changed, though the distribution of the inputs remains the same. In this case, the model can be simply retrained on the latest data. More research can be undertaken to explore how to adapt the model retraining process, e.g., selecting suitable training datasets based on the root causes of model performance issues.

Analyzing the performance of retrained models. In reality, not all model retraining rounds yield improved models. A retrained model may fail the A/B testing due to poor selection of retraining data, and issues can also arise from the inference service rather than the model itself. In such cases, the old model remains active, and human experts are required to investigate the reasons when using the CTCD-e MLOps pipeline. Automating the investigation process of A/B testing failures is a potential area for further research, enabling the pipeline to intelligently decide if (and when) it can autonomously trigger the next retraining iteration or if human experts need to be involved. More research could also be conducted to adapt the next retraining round based on the identified failure reasons.

Automatic thresholding. When using alerts as a retraining trigger, setting an appropriate threshold for triggering an alert is imperative. A too-sensitive threshold may trigger unnecessary retraining and waste computational resources, while a too-loose threshold may be unresponsive to model performance issues. Finding an appropriate threshold in a trial-and-error approach can be time-consuming and burdensome. Therefore, further research is needed to develop approaches for suggesting suitable thresholds based on business requirements and resource constraints.

C. Related Approaches

Online learning [15] is an alternative approach to CT for adapting ML models to dynamic production environments. Unlike CT, where models are fit with batch data offline and deployed online, online learning aims to update models immediately upon the arrival of each data point in a data stream. Our CTCD-e MLOps pipeline is not intended for online learning use cases. Instead, it is a complementary solution when online learning is insufficient to guarantee model performance, for example, due to the noise and outliers of incoming data.

Due to resource constraints, sometimes it may be impossible to use all historical data to retrain models, leading to the issue of catastrophic forgetting [16] when retraining deep neural network models. Catastrophic forgetting refers to the problem where deep neural network models forget previously learned information when being trained using the latest data. Continual learning (CL) [17] is a set of algorithms aiming to address catastrophic forgetting, and recent research has explored how to leverage CL in CT for models. Recent studies [18], [19] provide CT and CD for models, where data drift is used as the retraining trigger, and models are then trained using CL techniques. These two studies focus on neural network models and provide a CL library for prototyping and evaluating CL algorithms. Our CTCD-e MLOps pipeline is not limited to neural network models and allows users to implement various retraining algorithms, including CL, using Kubeflow Pipelines.

Active learning [20] is a set of techniques that select the most representative data samples from a massive dataset for manual labeling, thereby reducing the human effort of data labeling without decreasing the quality of model training. Incorporating active learning into the training data preparation may extend model CT to use cases requiring human annotators.

Besides model retraining, several approaches are available to address model performance degradation caused by data drift. For example, in the case of recurrent data drift, a pre-trained model trained on the data with a similar distribution of the drifted data can be retrieved from a model registry to replace the degraded one, thereby mitigating the effects of data drift without training a new model each time the drift occurs. Adaptive ensembles can combine the predictions of multiple models and dynamically adjust the ensemble composition based on the observed data drift pattern, thereby

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.

improving the overall prediction quality without retraining new models. To improve the efficiency of the CTCD-e MLOps pipeline, it would be valuable to incorporate these alternative approaches and develop automated techniques for selecting the most suitable approach to maintain prediction quality when data drift occurs.

D. Threats to Validity

Internal validity refers to whether our proposed MLOps pipeline can provide flexible CT and CD support for models. We carried out a proof-of-concept evaluation to validate the pipeline's capabilities. However, a limitation is that the pipeline has yet to be used in real-world use cases. The internal validity could be improved by conducting an empirical case study involving ML practitioners and real-world data usage with the pipeline.

External validity concerns whether our MLOps pipeline can be generalized to various use cases. To improve the external validity, we primarily utilized commonly used tools, as reflected in the number of stars received by the corresponding GitHub repositories. An exception is Iter8, which is less mature than other A/B testing tools, such as Flagger[13] and Argo Rollouts[14]. Despite its relative immaturity, we opted for Iter8 because of its unique feature of dynamic traffic splitting using novel algorithms. A threat to the external validity of our MLOps pipeline is its current limitation to use cases where ground truth is readily available and the data is tabular. To address this limitation, we intend to incorporate model monitoring approaches that can handle delayed ground truth in the future, as well as enable users to plug their monitoring algorithms into the pipeline.

VII. Conclusions

We presented an open-source MLOps pipeline that can autonomously adapt ML systems by continuous training and deployment (CT and CD). ML models are retrained based on monitored triggers, and the pipeline leverages automated A/B testing to perform continuous experimentation as a part of the deployment to select the better model for serving. We built the pipeline primarily using existing open-source components, with any non-existing components being relatively straightforward and released as open-source. Users can adopt and customize the pipeline to their use cases and ML systems without specific knowledge beyond MLOps pipelines and monitoring. The continuous experimentation loop brings forward new challenges for highly automated MLOps as future research directions.

In future work, it would be interesting to expand the evaluation of the pipeline to a wider range of models, particularly large models, which would provide a more comprehensive understanding of the time and resource utilization of model retraining and A/B testing. To further identify the potential areas of improvement, we consider it beneficial to conduct an empirical case study involving ML practitioners and systematically compare our pipeline with existing approaches. Moreover, active learning could be incorporated to expand the pipeline's applicability to use cases requiring human annotators. Incorporating a model registry and adaptive ensembles to maintain model performance against data drift could also be further explored to improve the pipeline's efficiency.

[13]https://flagger.app/

[14]https://argo-rollouts.readthedocs.io/en/stable/

References

[1] S. Mäkinen, H. Skogström, E. Laaksonen, and T. Mikkonen, "Who needs MLOps: What data scientists seek to accomplish and how can MLOps help?" in *IEEE/ACM Workshop on AI Engineering (WAIN)*, 2021.

[2] S. Alla and S. K. Adari, "What is MLOps?" in *Beginning MLOps with MLFlow: Deploy Models in AWS SageMaker, Google Cloud, and Microsoft Azure*. Apress, 2021, pp. 79–124.

[3] M. Steidl, M. Felderer, and R. Ramler, "The pipeline for the continuous development of artificial intelligence models—current state of research and practice," *Journal of Systems and Software*, vol. 199, 2023.

[4] Google. MLOps: Continuous delivery and automation pipelines in machine learning. [Online]. Available: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

[5] M. M. John, H. H. Olsson, and J. Bosch, "Towards mlops: A framework and maturity model," in *Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2021.

[6] M. Testi, M. Ballabio, E. Frontoni, G. Iannello, S. Moccia, P. Soda, and G. Vessio, "Mlops: A taxonomy and a methodology," *IEEE Access*, vol. 10, pp. 63 606–63 618, 2022.

[7] T. Diethe, T. Borchert, E. Thereska, B. Balle, and N. Lawrence, "Continual learning in practice," in *Workshop on Continual Learning*, 2018.

[8] D. Sato, A. Wider, and C. Windheuser. (2019) Continuous delivery for machine learning. [Online]. Available: https://martinfowler.com/articles/cd4ml.html

[9] J. Bright, A. Cerè, G. Schinas, and H. Theiss. (2021) Automate model retraining with amazon sagemaker pipelines when drift is detected. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/automate-model-retraining-with-amazon-sagemaker-pipelines-when-drift-is-detected/

[10] E. Raj, D. Buffoni, M. Westerlund, and K. Ahola, "Edge MLOps: An Automation Framework for AIoT Applications," in *IEEE International Conference on Cloud Engineering*, 2021, pp. 191–200.

[11] Y. Zhou, Y. Yu, and B. Ding, "Towards MLOps: A case study of ML pipeline platform," in *International Conference on Artificial Intelligence and Computer Engineering*, 2020.

[12] N. D. R and Mohana, "Jenkins pipelines: A novel approach to machine learning operations (mlops)," in *International Conference on Edge Computing and Applications*, 2022.

[13] J. Ereth, "Dataops-towards a definition." in *CEUR Workshop Proceedings*, vol. 2191, 2018.

[14] M. Toslali, S. Parthasarathy, F. Oliveira, H. Huang, and A. K. Coskun, "Iter8: Online experimentation in the cloud," in *ACM Symposium on Cloud Computing*, 2021.

[15] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, 2021.

[16] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends in cognitive sciences*, vol. 3, no. 4, pp. 128–135, 1999.

[17] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, "Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges," *Information fusion*, vol. 58, pp. 52–68, 2020.

[18] R. Semola, V. Lomonaco, and D. Bacciu, "Continual-learning-as-a-service (claas): On-demand efficient adaptation of predictive models," in *International Workshop on Pervasive Artificial Intelligence*, 2022.

[19] Y. Huang, H. Zhang, Y. Wen, P. Sun, and N. B. D. Ta, "Modelci-e: Enabling continual learning in deep learning serving systems," 2021. [Online]. Available: https://arxiv.org/pdf/2106.03122.pdf

[20] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, B. B. Gupta, X. Chen, and X. Wang, "A survey of deep active learning," *ACM Computing Surveys*, vol. 54, no. 9, 2021.

Authorized licensed use limited to: Lebanese American University. Downloaded on August 26,2025 at 19:08:34 UTC from IEEE Xplore. Restrictions apply.